Technical introduction to PSOC™ Edge E84 features

Chapter 6: Low power

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 6 discusses PSOC™ Edge's low-power features.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
3 Code example: Power measurements **5**
3.1 Objective 5
3.2 Description 5
3.3 Flow chart 6
3.4 Hardware overview 6
3.5 Project creation 7
3.6 Output 11
3.7 Modify the application 12
3.7.1 Change system power mode 12
3.8 Conclusion 14
4 Code example: Switching power modes **15**
4.1 Objective 15
4.2 Description 15
4.3 Hardware overview 15
4.4 Project Creation 16
4.5 Output 17
4.6 Modify the application 19
4.6.1 Change the CPU frequency 19
4.7 Conclusion 22
5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **23**
6 Appendix B: KIT_PSE84_EVAL details **26**
Revision history **27**
Disclaimer **28**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run several application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

3 Code example: Power measurements

3.1 Objective

This code example demonstrates how to configure and run low-power benchmarks for various PSOC™ Edge E84 MCU power modes such as Sleep, Deep Sleep, and Hibernate. Power measurements are done with different clock and power mode configurations matching the datasheet specifications.

3.2 Description

The PSOC™ Edge E84 datasheet includes power specifications in various configurations of power mode, frequency, memories, and so on. Part of these specifications are shown in the following figure.

Table 16 CPU currents, active modes, battery powered configuration, VBAT=3.3 V, VDDD=1.8 V

Spec ID	Description	CM33 (MHz)	CM55 (MHz)	IBAT		IDDD		Units
				Typical	Max 70°C	Typical	Max 70°C	
System HP (High Performance) mode (See condition details above), VCCD = 0.9 V								
PD0 = On (Low Power Domain), PD1 = Off (High Performance Domain) (See condition details above)								
CM33 CPU Sleep (RRAM)								
SIDH00A		200	-	1.9	5.5	0.8	1.2	mA
SIDH00B		70	-	1.2	4.7	0.8	1.2	mA
CM33 Active, execute from RRAM, RRAM enabled								
SIDH01A		200	-	4.0	7.7	0.8	1.2	mA
SIDH01B		70	-	1.9	5.5	0.8	1.2	mA

Figure 1 Excerpt from PSOC™ Edge E84 datasheet

Note that PSOC™ Edge can be configured in two power distributions.

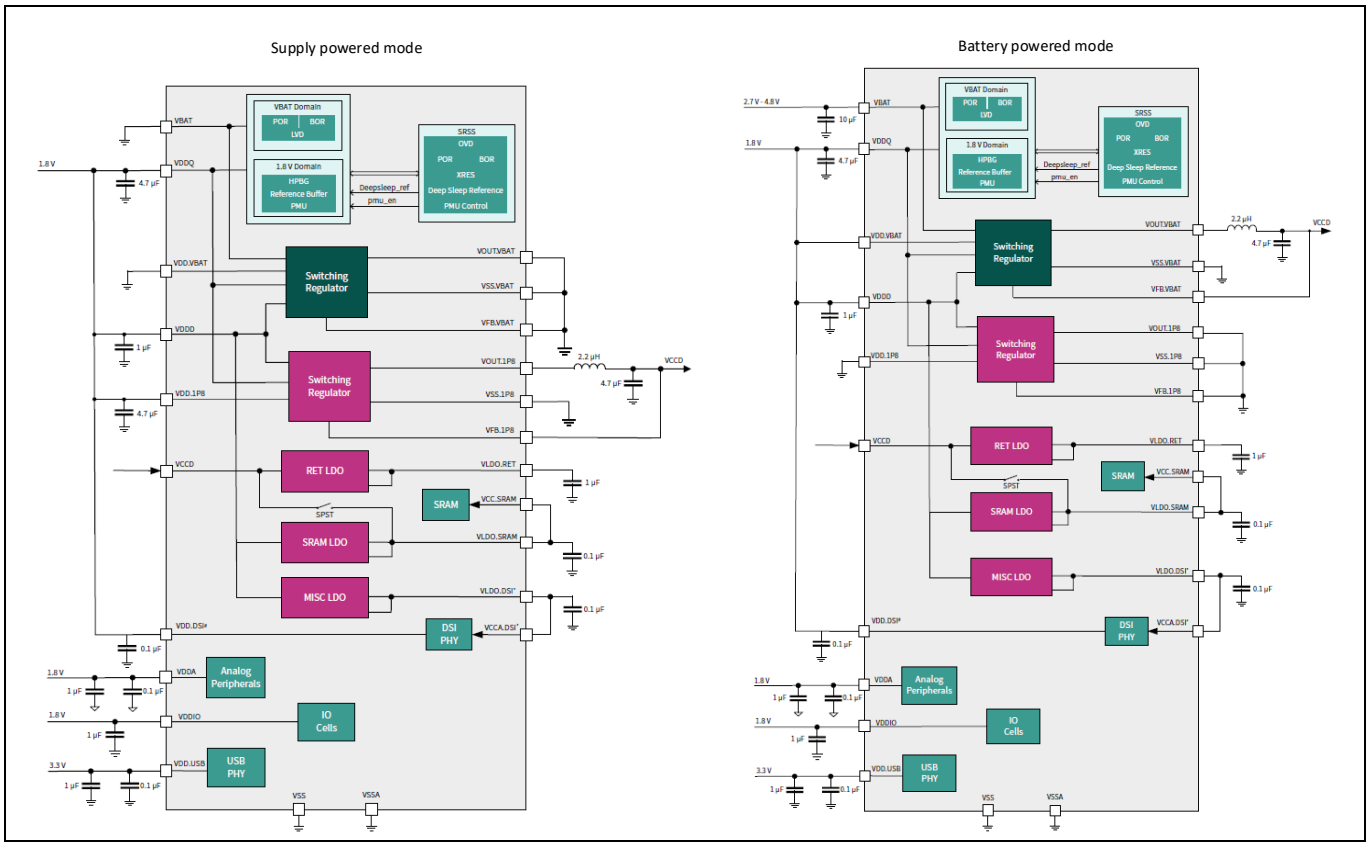


Figure 2 Power distribution modes for PSOC™ Edge

- Supply powered, only requires 1.8 V
- Battery powered, uses a battery (2.7 V-4.8 V) and 1.8 V

The SOM board included in the PSOC™ Edge EVK is configured in battery-powered mode, and thus, this configuration will be used for this training.

The code example used for this lab configures the clock frequencies and system power modes of the PSOC™ Edge E84 MCU for many of the configurations mentioned in the datasheet. You can select the desired configuration in the *specs.h* file.

After building and programming the application, you can measure the current consumed by the PSOC™ Edge E84 MCU. The code example uses both the Arm® Cortex®-M33 CPU (CM33) and the Arm® Cortex®-M55 CPU (CM55) to run the benchmark.

3.3 Flow chart

In this code example, resource initialization is performed by the CM33 non-secure (CM33_NS) code. It configures the system clocks, pins, clock-to-peripheral connections, and other platform resources. It then enables the CM55 core using the Cy_SysEnableCM55() function.

The following flow chart explains the complete code flow of this code example:

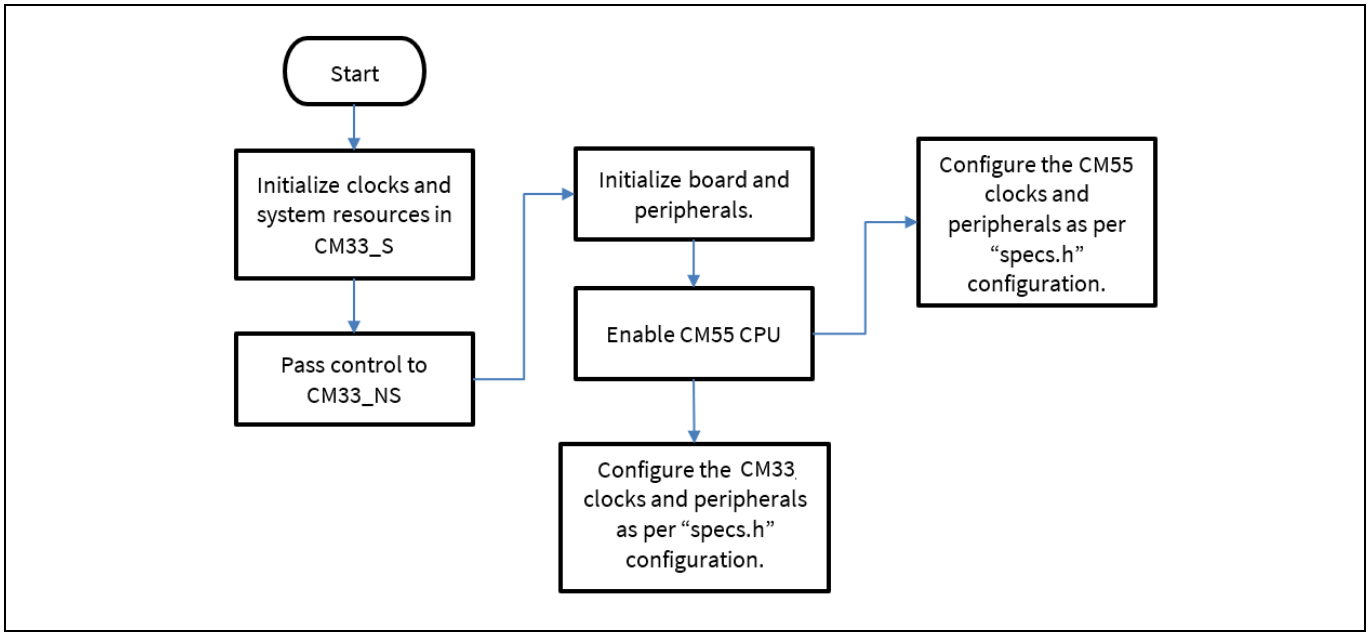


Figure 3 Flow diagram of power measurements example

3.4 Hardware overview

As mentioned in Description, the SOM board included with the PSOC™ Edge EVK is pre-configured in battery-powered mode, meaning that the device has a 3.3 V component (VBAT), and a 1.8 V component (VDDD, VDDQ, VDDIO, VCCD, and so on) .

To measure the two power rails, we will use J25 and J26.

- J25 measures IDDD/IDDIO (1.8 V)
- J26 measures IBAT (3.3 V)

Code example: Power measurements

Note: The code example README suggests measuring IDDD on FB1 and performing some hardware modifications to remove extra current from EVK. These suggestions give a more accurate measurement; however, they are not performed in this training since J25 provides an easier measurement for training purposes.

Note: This example requires BOOT SW in OFF position.

See

Appendix B: KIT_PSE84_EVAL details for more details.

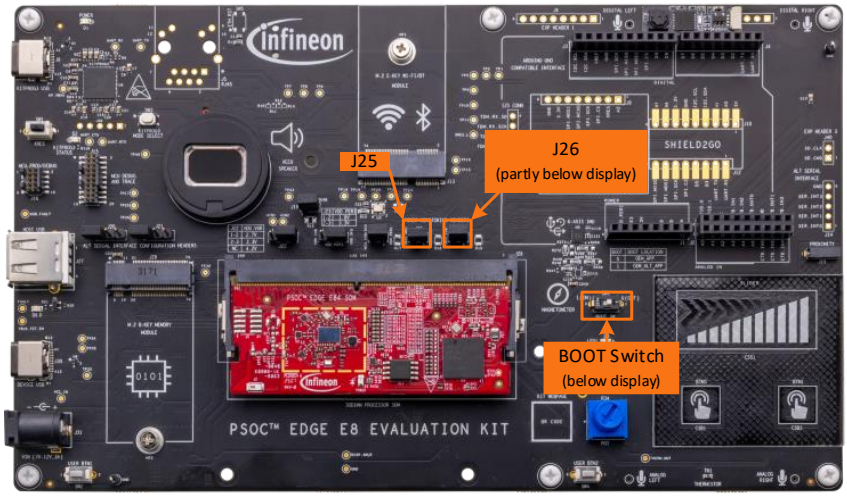


Figure 4 Hardware diagram of power measurements example

3.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Power Measurements** application under the **Peripherals** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

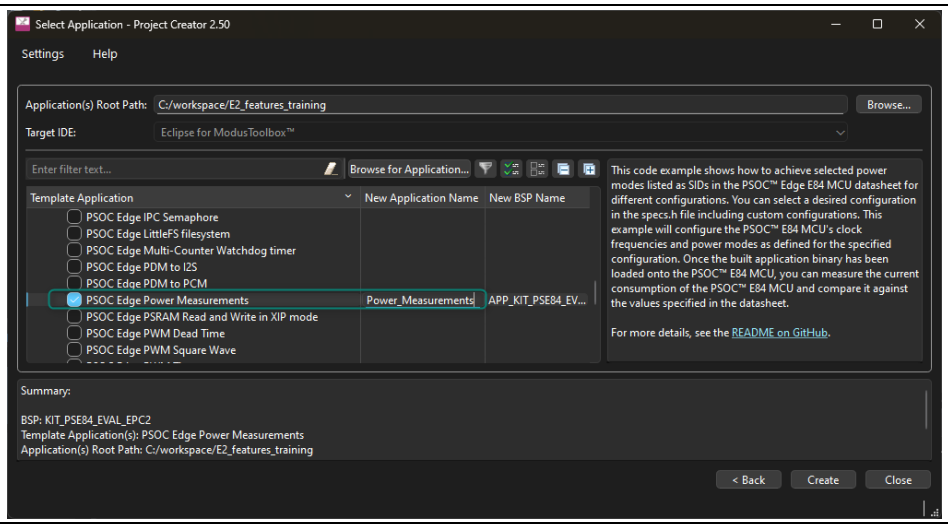


Figure 5 Creating power measurements example

3. This code example supports different power configurations, but let's review the default device configuration in **Device Configurator**.

   a) Open Device Configurator from the Quick Panel.

   b) Go to the **System** panel and select **System→Power** > **Power**.

   As observed below, the default settings set the device in High-performance (HP) mode which allows running the CM33 CPU up to 200MHz and the CM55 CPU up to 400MHz. The settings also show that System Deep Sleep is the default when going to low-power mode.

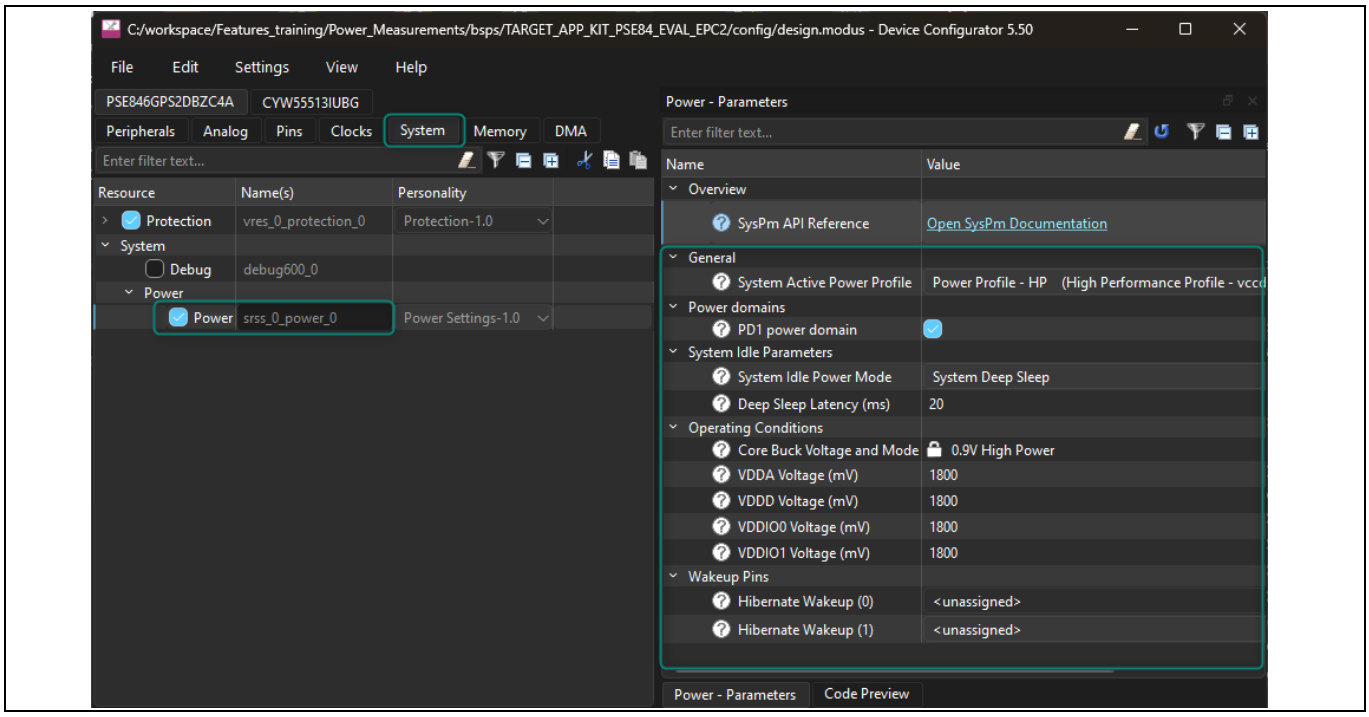


Figure 6 Default system configuration

   c) Now go to the **Clocks** panel.

   As observed, the system uses DPLL_LP0 at 400 MHz as the source for CM33 (CLK_HF0) and for CM55 (CLK_HF1); however, CLK_HF0 has a divider to avoid exceeding its maximum frequency.

   This view also shows the configuration of other clocks such as the other 2 DPLLs or CLK_HF8, which are disabled by default.

   The configuration for this code example is not necessarily the same as other examples. It's important for a low-power application to configure clocks to the minimum viable frequency or disable them when unused.

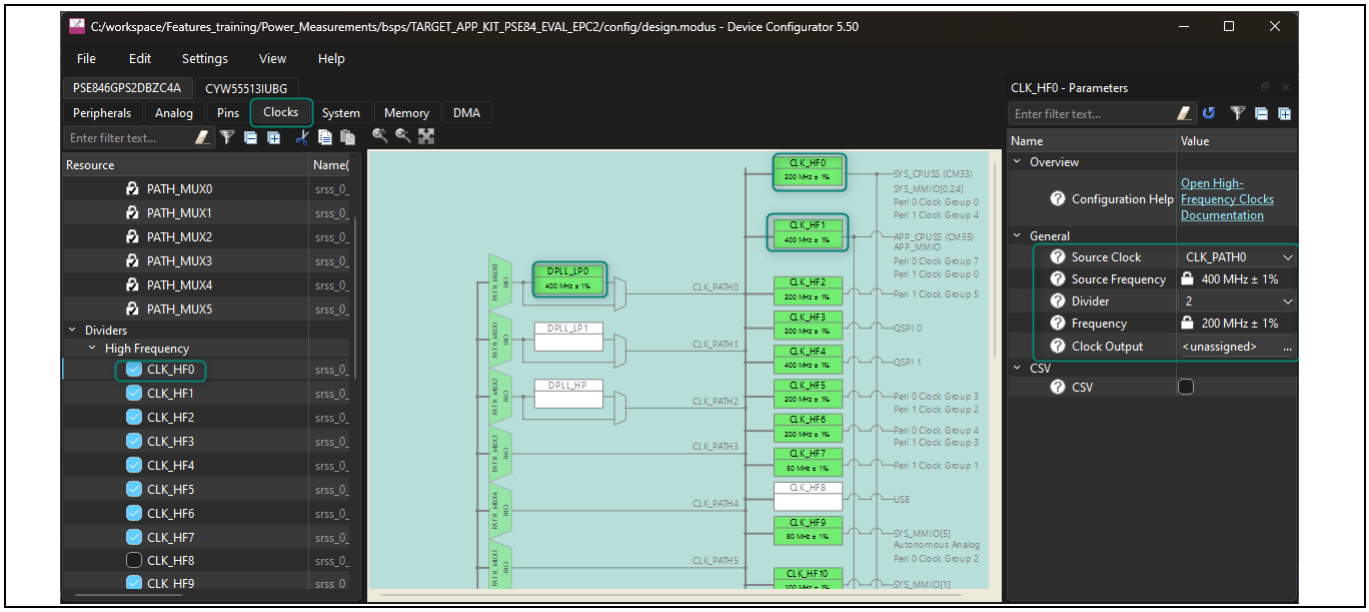


Figure 7 Default clock configuration of the device

d) Now go to the **Peripherals** tab. Observe how this example disables most peripherals by default. It's important to disable disable unnecessary peripherals when developing low-power applications.

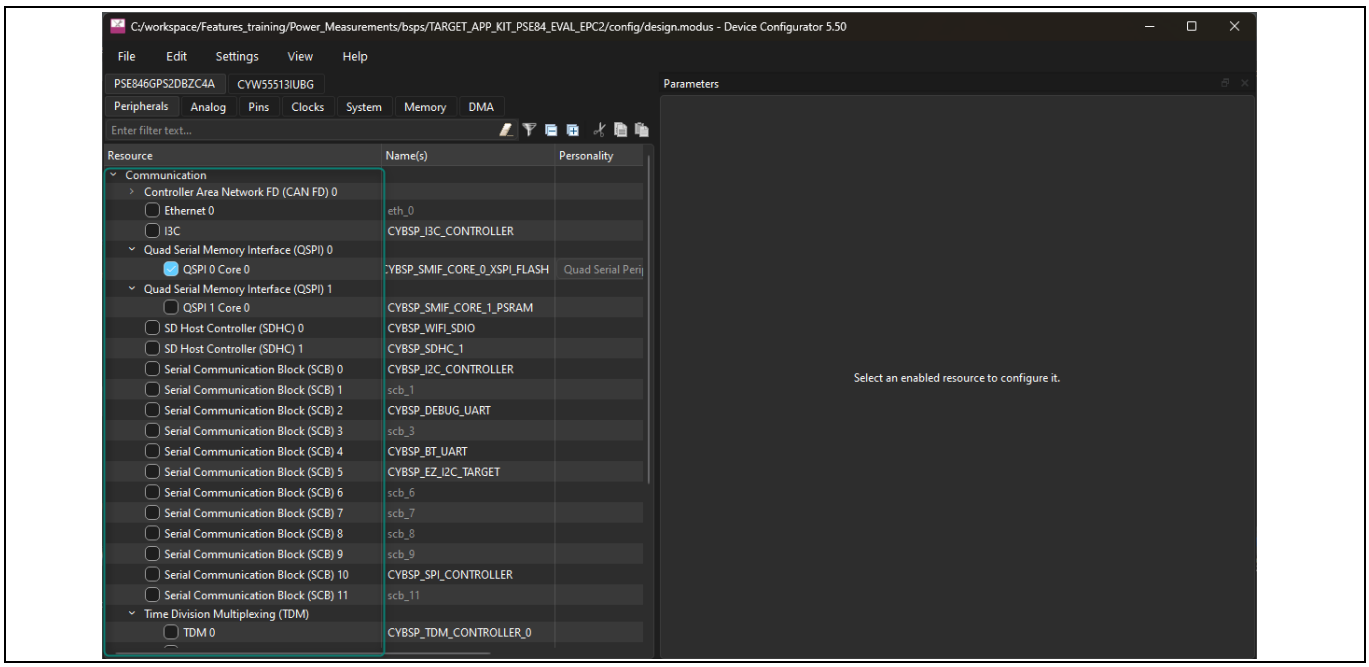


Figure 8 Default configuration of peripherals

e) Lastly, go to the **Pins** tab and observe how this application only initializes minimal GPIOs. It's important for applications to terminate pins properly to reduce power consumption.
The recommendation for PSOC™ Edge is that unused GPIOs should be configured in high-impedance drive mode with input buffer disabled, which is the default pin reset state.

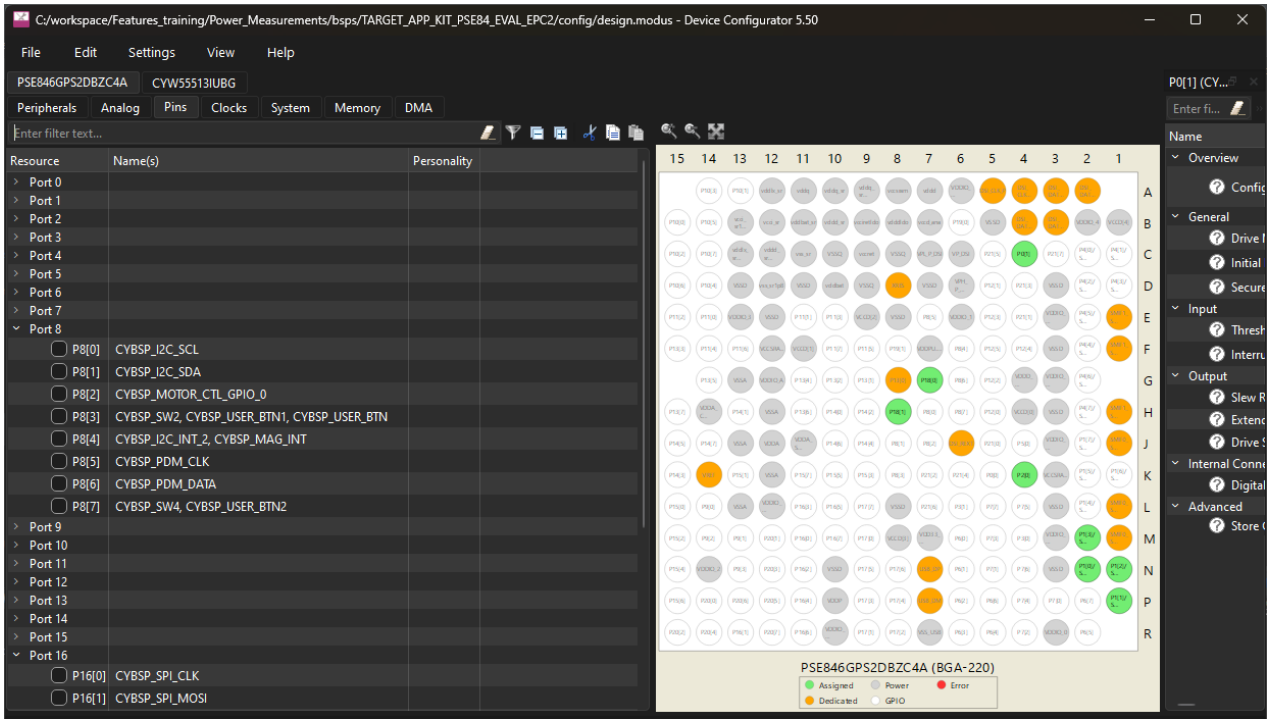
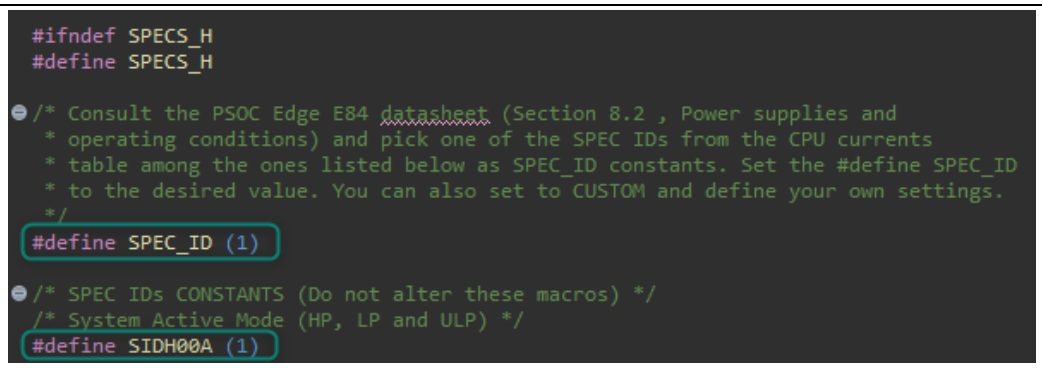


Figure 9 Default pin configuration

4. Now, let's analyze the default settings of the code example.

   a) Open *shared\include\specs.h*

   b) Observe the definition of **SPEC_ID** which is set to 1, corresponding to **SIDH00A**.
   This configuration correlates to the datasheet spec SIDH00A, which, as observed below, is for the CM33 in sleep mode at 200 MHz, with the CM55 off.



```
#ifndef SPECS_H
#define SPECS_H

/* Consult the PSOC Edge E84 datasheet (Section 8.2 , Power supplies and
 * operating conditions) and pick one of the SPEC IDs from the CPU currents
 * table among the ones listed below as SPEC_ID constants. Set the #define SPEC_ID
 * to the desired value. You can also set to CUSTOM and define your own settings.
 */
#define SPEC_ID (1)

/* SPEC IDs CONSTANTS (Do not alter these macros) */
/* System Active Mode (HP, LP and ULP) */
#define SIDH00A (1)
```

Table 16 CPU currents, active modes, battery powered configuration, VBAT=3.3 V, VDDD=1.8 V

Spec ID	Description	CM33 (MHz)	CM55 (MHz)	IBAT		IDDD		Units
				Typical	Max 70°C	Typical	Max 70°C	
System HP (High Performance) mode (See condition details above), VCCD = 0.9 V								
PD0 = On (Low Power Domain), PD1 = Off (High Performance Domain) (See condition details above)								
CM33 CPU Sleep (RRAM)								
SIDH00A		200	-	1.9	5.5	0.8	1.2	mA
SIDH00B		70	-	1.2	4.7	0.8	1.2	mA

Figure 10 Default configuration of power measurements example

5. Build and program the application.

3.6 Output

After programming, the application starts automatically.

Use J25 and J26 to measure the current consumption with a multimeter.

Table 1 Power calculation

Voltage (V) in volts	Current (I) in amps	Power (P) in watts
VDDD = 1.8 V	I_{DDD} = measure across J26	P_{DDD} = VDDD x I_{DDD}
VBAT = 1.8 V	I_{BAT} = measure across J25	P_{BAT} = VBAT x I_{BAT}

a) Total power = PDDD + PBAT

b) The EVK includes additional currents for external circuitry or the VDDIOx supplies. See code example README for more details.

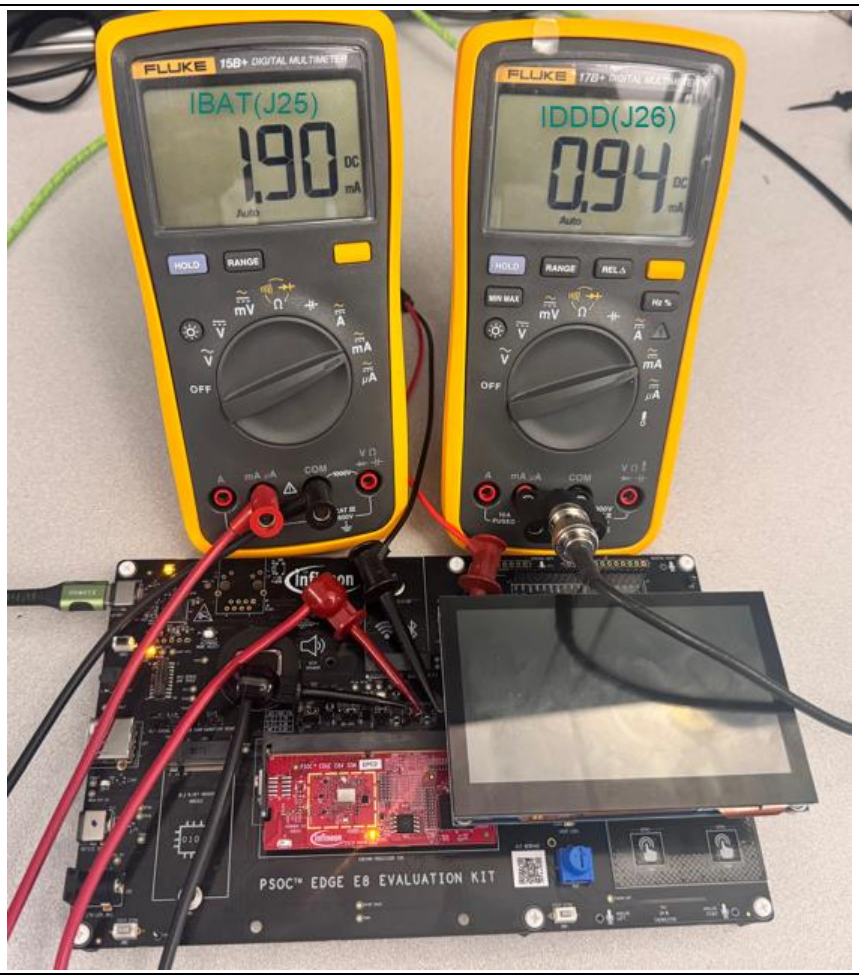


Figure 11 Output of power measurements example using default configuration

The power consumption should match the specification for SIDH00A, which sets CM33 in Sleep mode and the CM55 off. However, it's important to note that the EVK includes some additional currents.

The code example README recommends some EVK modifications to reduce power consumption.

3.7 Modify the application

3.7.1 Change system power mode

Try testing other power configurations.

Hints for the advanced modification

- *Specs.h* includes definitions to configure the power modes

Solution

6. As observed previously, the example is configured to test SIDH00A by default; however, it's possible to test other configurations by simply modifying **SPEC_ID**.
7. Modify *shared\include\specs.h* as shown below to test Hibernate mode.

Code listing 1 Modification to specs.h to test hibernate mode

```
/* Consult the PSOC™ Edge E84 datasheet (Section 8.2 , Power supplies and
 * operating conditions) and pick one of the SPEC IDs from the CPU currents
 * table among the ones listed below as SPEC_ID constants. Set the #define SPEC_ID
 * to the desired value. You can also set to CUSTOM and define your own settings.
 */
#define SPEC_ID (20)
```

8. Rebuild, reprogram, and measure the power consumption.
   Note that the example is configured to match SIDHIBA, which configures the device in System Hibernate using no clocks.
   As noted previously, the EVK includes some additional currents which are not considered in the datasheet; however, our measurement should be close.
   The code example README recommends some EVK modifications to reduce power consumption.

Table 17 CPU currents, deep sleep and hibernate modes, battery powered configuration, VBAT=3.3 V, VDDD=1.8 V

Spec ID	Description	IBAT		IDDD		Units
		Typical	Max 70°C	Typical	Max 70°C	
Hibernate Mode - VDDIO currents not included						
SIDHIBA	No Clocks	0.3	0.9	1.6	16.5	µA

Figure 12 Datasheet specification SIDHIBA for hibernate mode



Figure 13 Output after modifying example to use hibernate mode

9. Try other macros in *spec.h*.
   Observe that a CUSTOM (21) configuration can also be used, allowing for more flexibility in the configuration.

```
#elif(SPEC_ID == CUSTOM)
#define DPLL_FREQ_LP_HZ            (140000000U)

/* Options for the system power mode:
 * SYSTEM_HP: High-performance mode for maximum processor speed
 * SYSTEM_LP: Low-power mode for reduced processor speed and power consumption
 * SYSTEM_ULP: Ultra-low-power mode for minimal power consumption
 */
#define SYSTEM_HP

/* Option to disable CM55 core:
 * Uncomment the below macro to disable CM55 core.
 */
/* #define DISABLE_CM55 */

/* Option to disable Power Domain1 (PD1):
 * Uncomment the below macros to disable Power Domain 1.
 */
/* #define DISABLE_PD1 */
/* #define CM55_DEEP_SLEEP */

/* Options for the CM33 super loop:
 * CM33_DHRYSTONE: Run Dhrystone benchmark on CM33 CPU
 * CM33_WHILE_LOOP: Run infinite loop and keep CM33 CPU active
 * CM33_SLEEP: Put CM33 CPU in low-power sleep mode
 * CM33_DEEP_SLEEP: Put CM33 CPU in deep sleep mode
 * CM33_HIBERNATE: Put the System to Hibernate mode
 */
#define CM33_DHRYSTONE

/* Options for the CM55 super loop. CM55 should be in deepsleep
 * mode and should not be altered when PD1 is disabled
 * in above settings:
 * CM55_DHRYSTONE: Run Dhrystone benchmark on CM55 CPU
 * CM55_WHILE_LOOP: Run infinite loop and keep CM55 CPU active
 * CM55_SLEEP: Put CM55 CPU in low-power sleep mode
 * CM55_DEEP_SLEEP: Put CM55 CPU in deep sleep mode
 */
#if !defined(DISABLE_PD1)
#define CM55_DHRYSTONE

#endif
#endif
```

Figure 14 Custom configuration on specs.h

3.8 Conclusion

We developed a power measurement application for the PSOC™ Edge E84 MCU using ModusToolbox™. To achieve this, we configured the CM33 and CM55 in different power modes to measure power consumption.

4 Code example: Switching power modes

4.1 Objective

This code example demonstrates how to transition between the following system power modes in PSOC™ Edge E84 MCU:

- System High Performance (HP)
- System Low Power (LP)
- System Ultra Low Power (ULP)
- System Deep Sleep
- System Hibernate

This code example has a three-project structure: CM33 Secure, CM33 non-secure, and CM55. Extended Boot launches the CM33 Secure project present in RRAM from a fixed location, which then configures the external QSPI flash in XIP mode and launches the CM33 non-secure application. Additionally, CM33 non-secure application enables the CM55 CPU and launches the CM55 application in external flash.

4.2 Description

In this code example, resource initialization is performed by this CM33 non-secure project. It configures the system clocks, pins, clock-to-peripheral connections, and other platform resources. It then enables the CM55 core using the Cy_SysEnableCM55() function.

In the CM33 non-secure project, the clocks and system resources are initialized by the BSP initialization function.

The table below displays the configurations for different system power modes. It includes the VCCD voltage, the CM33 and the CM55 clock speeds.

Table 2 System power modes

Power mode	VCCD	CM33 max clock	CM55 max clock
HP (System High Performance)	0.9 V	200 MHz	400 MHz
LP (System Low Power)	0.8 V	70 MHz	140 MHz
ULP (System Ultra Low Power)	0.7 V	50 MHz	50 MHz
DS (System Deep Sleep)	0.7 V	Off	Off
System Hibernate	Off	Off	Off

4.3 Hardware overview

As mentioned in Description, the SOM board included with the PSOC™ Edge EVK is pre-configured in battery-powered mode, meaning that the device has a 3.3 V component (VBAT), and a 1.8 V component (VDDD, VDDQ, VDDIO, VCCD, and so on) .

To measure the two power rails, we will use J25 and J26.

- J25 measures IDDD/IDDIO (1.8 V)
- J26 measures IBAT (3.3 V)

Note: The power measurements code example README suggests measuring IDDD on FB1 and performing some hardware modifications to remove extra current from EVK. These suggestions give a more accurate measurement; however, they are not performed in this training since J25 provides an easier measurement for training purposes.

Note: This example requires BOOT SW in OFF position.

See

Appendix B: KIT_PSE84_EVAL details for more details.

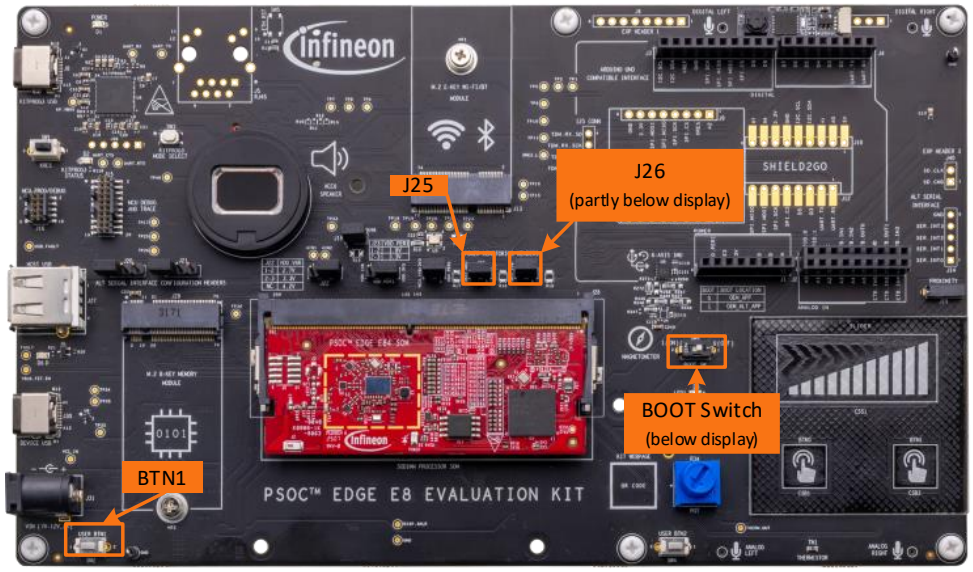


Figure 15 Hardware diagram for switching power modes application

4.4 Project Creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Switching Power Modes** application under the **Peripherals** section.
   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

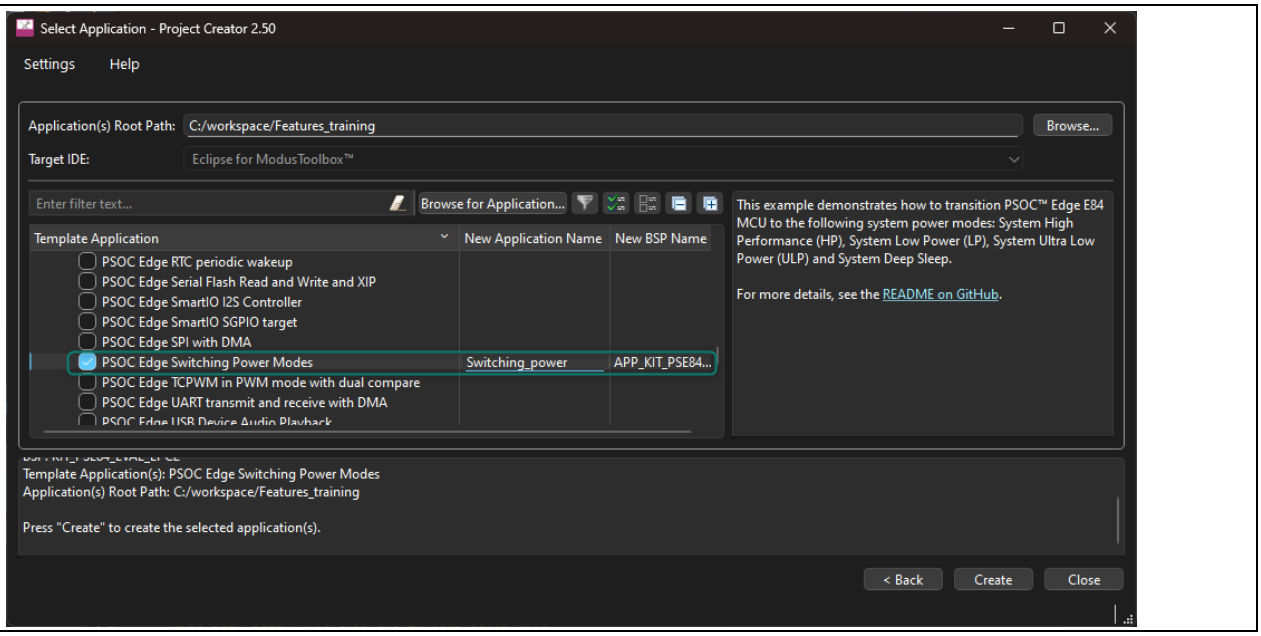


Figure 16 Creating switching power modes example

3. Build and program the application.

4.5 Output

After programming, the application automatically starts. To change the power mode, press USER BTN1 on the EVK. Measure the current consumption and note the difference in power consumption when the system changes power modes.

The following flow chart explains the complete code flow of this code example.

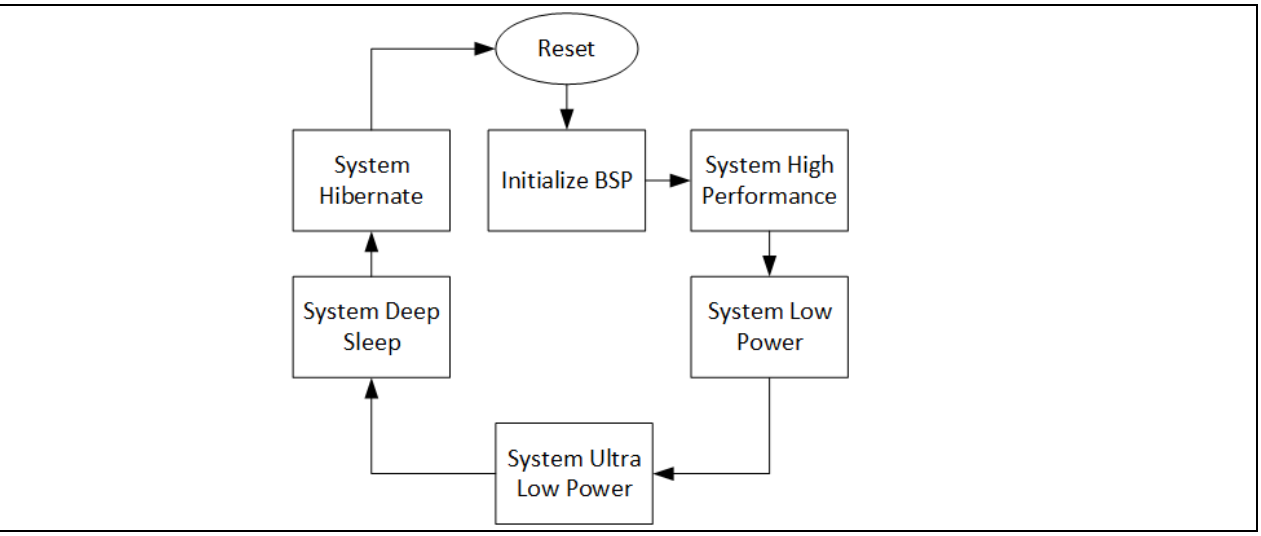


Figure 17 Flow diagram of application

Code example: Switching power modes

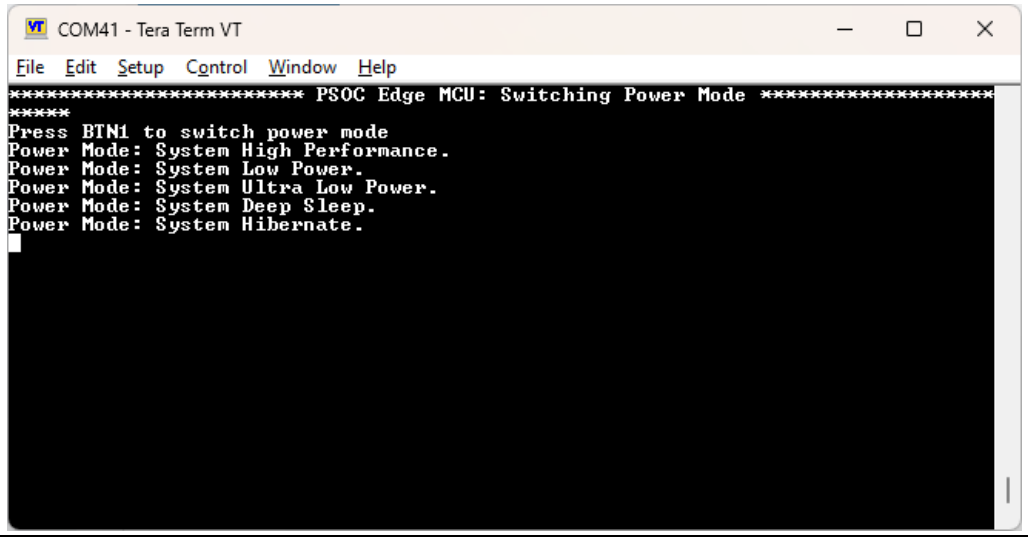


Figure 18 Output of switching power modes example



Figure 19 Power measurement of switching power modes application

4.6 Modify the application

4.6.1 Change the CPU frequency

The default code example is configuring the CM33 at 200 MHz in HP mode, 70 MHz in LP mode, and 50 MHz in ULP mode.

Try modifying the code to run the CM33 CPUI at 100 MHz in HP mode and 25 MHz in ULP mode.

Hints for the advanced modification

- Use Device Configurator to check the clock configuration of the CPU
- Modify source code to configure clocks properly

Solution

1. The **Device Configurator** provides an easy way to observe the default clock configuration.
   Open **Device Configurator** from the Quick Panel and observe that the CPU is driven by CLK_HF0, which is derived from *DPLL_LP0/CLK_PATH0*.
   The DPLL_LP0 is configured at 400 MHz, while the CLK_HF0 has a divider of 2, resulting in 200 MHz for the CM33 CPU.

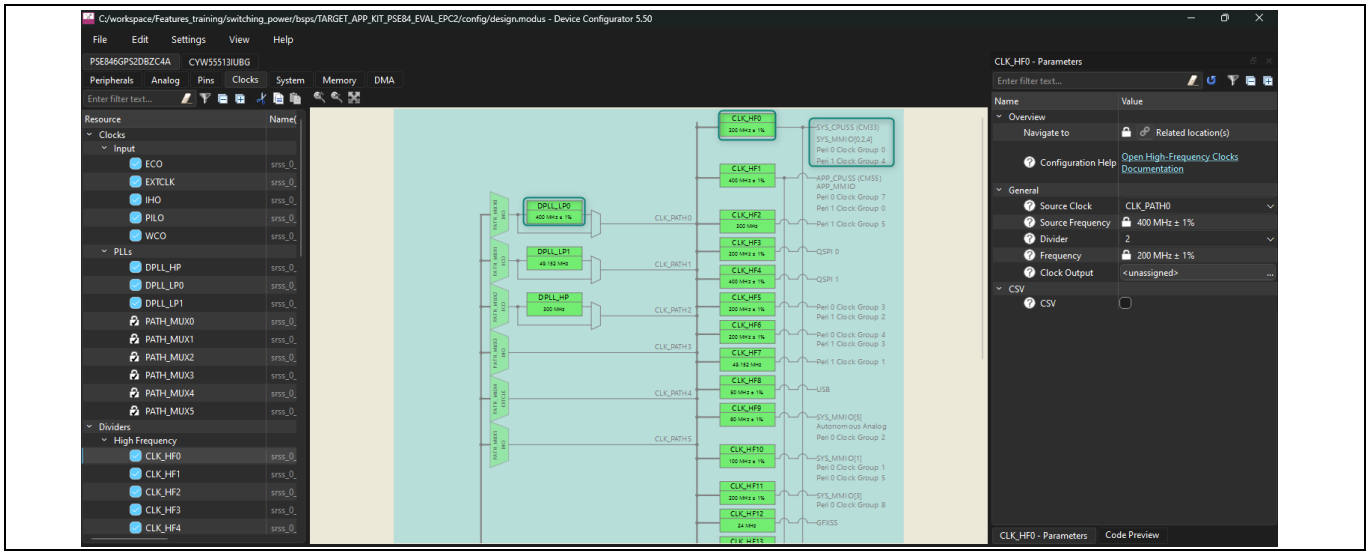


Figure 20 Default clock configuration of switching power modes example

2. Now, let's observe how the code example is configuring the clocks in each mode.
   Open *cm33_ns/main.c* and observe the device configuration when HP mode is selected as shown in Figure 12:

```
switch (next_power_state)
{
    case HIGH_PERFORMANCE:
    {
        /** Enter High Performance Power (HP) mode.*/
    a)  status = Cy_SysPm_SystemEnterHp();

        if (CY_SYSPM_SUCCESS != status)
        {
            handle_app_error();
        }// if CY_SYSPM_SUCCESS != status

        /** Check if the system successfully entered HP mode. */
        if (Cy_SysPm_ReadStatus() & CY_SYSPM_STATUS_SYSTEM_HP)
        {
            /** Set the RRAM to HP voltage mode*/
    b)      Cy_RRAM_SetVoltageMode(RRAMC0, CY_RRAM_VMODE_HP);

    c)      Cy_SysClk_ClkHfSetDivider(CY_CFG_SYSCLK_CLKHF0, CY_SYSCLK_CLKHF_DIVIDE_BY_2);

    d)      dpll_lp_set_freq(DPLL_FREQ_HP_HZ);

            /** Set the peripheral clock divider for the debug UART */
    e)      Cy_SysClk_PeriPclkSetDivider((en_clk_dst_t)CYBSP_DEBUG_UART_CLK_DIV_GRP_NUM,
                    CY_SYSCLK_DIV_16_BIT, 1U, UART_HP_DIV);
        }// if Cy_SysPm_ReadStatus & ...

        printf("Power Mode: System High Performance.\r\n");
        WAIT_FOR_TX_COMPLETE();
        next_power_state = LOW_POWER;
        break;
    }
```

Figure 21 Routine to enable HP mode in main.c

a) The system is placed into HP mode by calling *Cy_SysPm_SystemEnterHp*.

b) The RRAM voltage is updated by calling *Cy_RRAM_SetVoltageMode*.

c) *Cy_SysClk_ClkHfSetDivider* is called to set the CPU clock divider to 2 .

d) The local function dpll_lp_set_freq is called to set the DPLL frequency.

e) Note that *DPLL_FREQ_HP_HZ* is set to 400 MHz (resulting in CPU frequency of 200 MHz after the divider).

f) Modifying the DPLL will also affect the UART clock, thus *Cy_SysClk_PeriPclkSetDivider* is used to configure the UART divider.

3. In *cm33_ns/main.c*, modify the following code:

Code listing 2 Change CM33 frequency to 100 MHz in HP mode

```
/** Check if the system successfully entered HP mode. */
if (Cy_SysPm_ReadStatus() & CY_SYSPM_STATUS_SYSTEM_HP)
{
    /** Set the RRAM to HP voltage mode*/
    Cy_RRAM_SetVoltageMode(RRAMC0, CY_RRAM_VMODE_HP);

    Cy_SysClk_ClkHfSetDivider(CY_CFG_SYSCLK_CLKHF0, CY_SYSCLK_CLKHF_DIVIDE_BY_4);

    dpll_lp_set_freq(DPLL_FREQ_HP_HZ);

    /** Set the peripheral clock divider for the debug UART */
    Cy_SysClk_PeriPclkSetDivider((en_clk_dst_t)CYBSP_DEBUG_UART_CLK_DIV_GRP_NUM,
```

Code example: Switching power modes

```
     CY_SYSCLK_DIV_16_BIT, 1U, UART_HP_DIV);
}
printf("Power Mode: System High Performance at %uMHz\r\n",
        Cy_SysClk_ClkHfGetFrequency(CY_CFG_SYSCLK_CLKHF0)/1000000);
WAIT_FOR_TX_COMPLETE();
```

Note: Another option is to modify the DPLL0 frequency. This is optional; however, note that the UART clock divider must also be updated since the UART clock is derived from DPLL.

4. Now, divide the ULP clock as shown below:

Code listing 3 Change CM33 frequency to 25 MHz in ULP mode

```
case ULTRA_LOW_POWER:
{
    /** Disable the high-performance DPLL and Directly clock
     * from IHO (50 MHz)*/

    Cy_SysClk_PllDisable(SRSS_DPLL_LP_0_PATH_NUM);

    /** Enter Ultra-Low Power (ULP) mode.*/
    status = Cy_SysPm_SystemEnterUlp();
    if (CY_SYSPM_SUCCESS != status)
    {
        handle_app_error();
    }

    /** Check if the system successfully entered ULP mode. */
    if (Cy_SysPm_ReadStatus() & CY_SYSPM_STATUS_SYSTEM_ULP)
    {
        /** Set the RRAM to ULP voltage mode for lower consumption. */
        Cy_RRAM_SetVoltageMode(RRAMC0, CY_RRAM_VMODE_ULP);

        /** Set the high-frequency clock (CLKHF) to no divide */
        Cy_SysClk_ClkHfSetDivider(CY_CFG_SYSCLK_CLKHF0,
CY_SYSCLK_CLKHF_DIVIDE_BY_2);

        /** Set the peripheral clock divider for the debug UART */
        Cy_SysClk_PeriPclkSetDivider(
(en_clk_dst_t)CYBSP_DEBUG_UART_CLK_DIV_GRP_NUM, CY_SYSCLK_DIV_16_BIT, 1U,
UART_ULP_DIV);
    }
    printf("Power Mode: System Ultra Low Power at %uMHz\r\n",
       Cy_SysClk_ClkHfGetFrequency(CY_CFG_SYSCLK_CLKHF0)/1000000);
    WAIT_FOR_TX_COMPLETE();
```

```
    next_power_state = SYSTEM_DEEP_SLEEP;
    break;
}
```

Note: The DPLL is disabled in ULP mode and CLK_HF0 is driven directly by the internal high frequency oscillator (IHO) at 50 MHz.

5. Rebuild, reprogram and test the results.

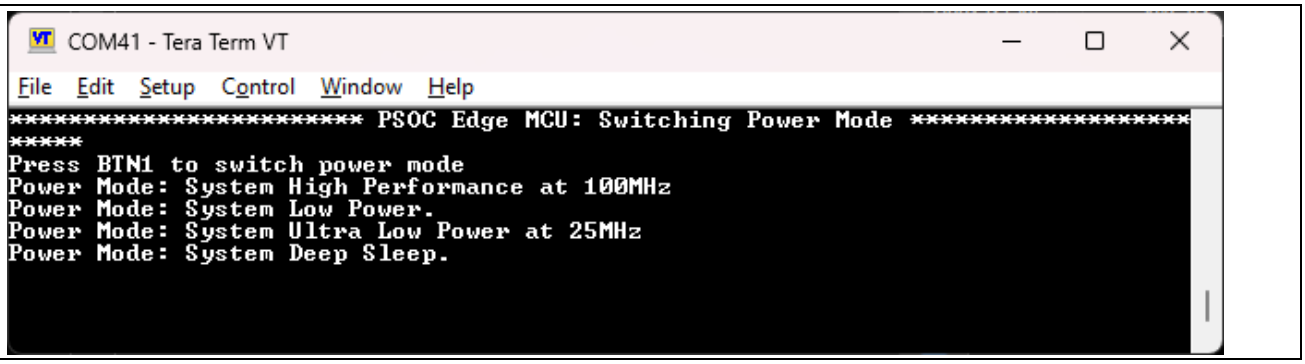


Figure 22 Output after changing HP and ULP frequencies

4.7 Conclusion

We have successfully developed a PSOC™ Edge E84 MCU application to switch power modes using ModusToolbox™. We can measure the power consumption in each mode.

Dynamic voltage and frequency scaling (DVFS) is a power management technique used in modern microcontrollers, including the PSOC™ Edge MCU, to optimize power consumption and performance. The concept of DVFS involves dynamically adjusting the voltage and frequency of the processor to match the required performance level, thereby reducing power consumption and heat generation.

Key principles of DVFS:

- **Voltage reduction**: Lowering the voltage supply to the processor reduces power consumption, as power is proportional to the square of the voltage
- **Frequency reduction**: Decreasing the processor frequency reduces power consumption, as power is proportional to the frequency
- **Voltage-frequency correlation**: To ensure stable operation, the frequency must be reduced before reducing the voltage, and the voltage must be increased before increasing the frequency. This is because a lower frequency requires a lower voltage to maintain stability, and a higher frequency requires a higher voltage to ensure reliable operation

Why is this sequence important?

If the voltage is reduced before reducing the frequency, the MCU may become unstable or even fail to operate correctly. Similarly, if the frequency is increased before increasing the voltage, the processor may stop working or even damage due to excessive voltage stress.

5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisites section.

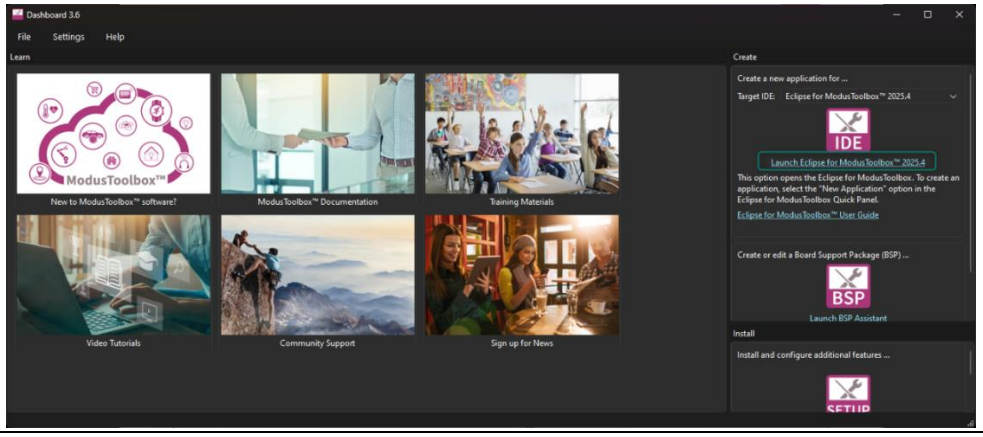


Figure 23 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

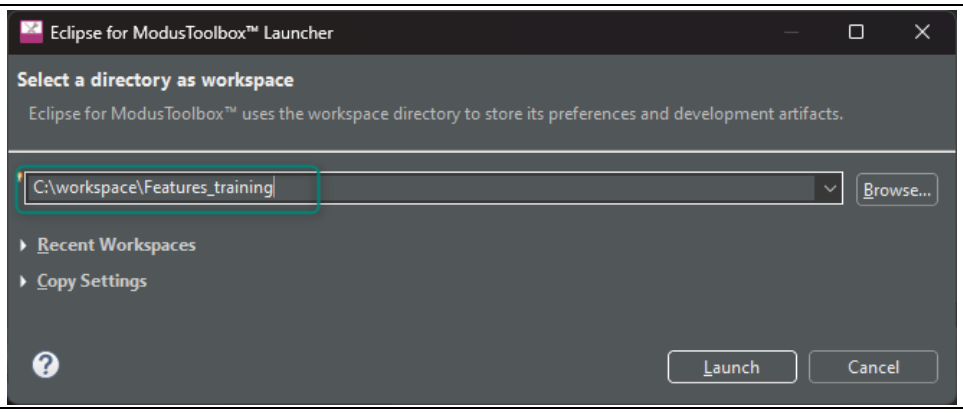


Figure 24 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

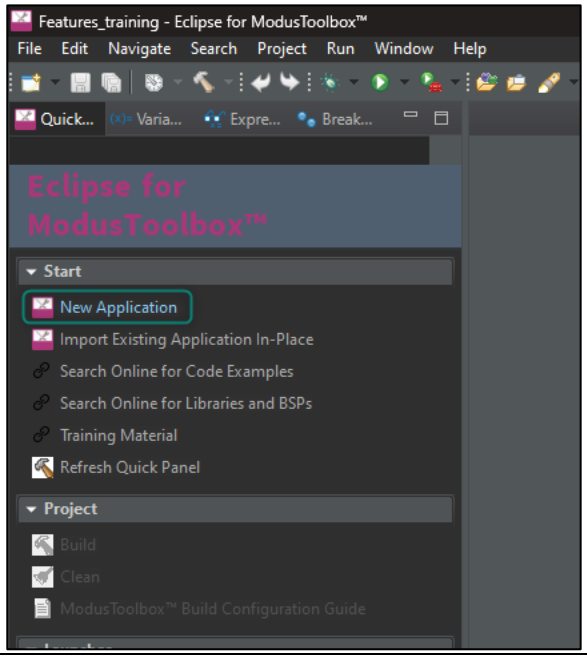


Figure 25 Create new application

4. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP). Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.

   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*

   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

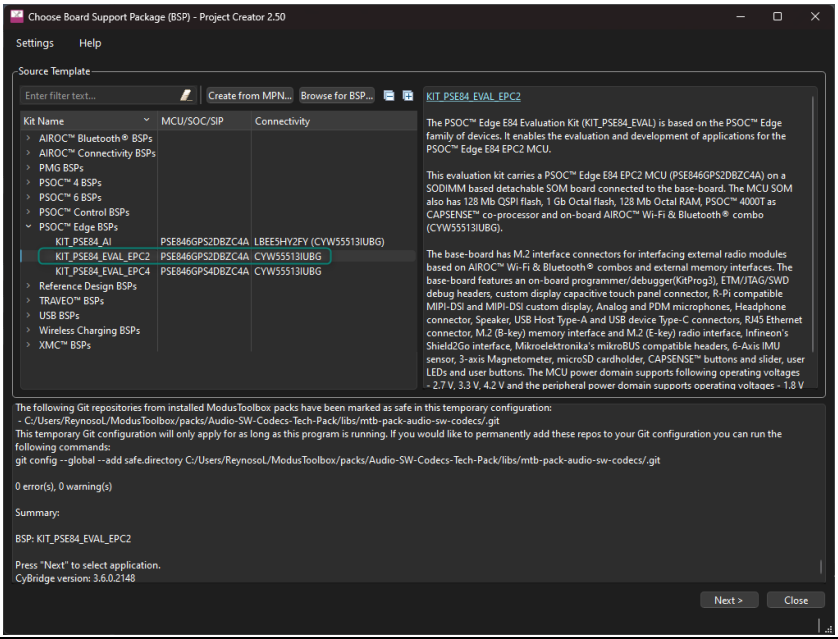


Figure 26 Selecting BSP in Project Creator

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.

   Figure 27 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

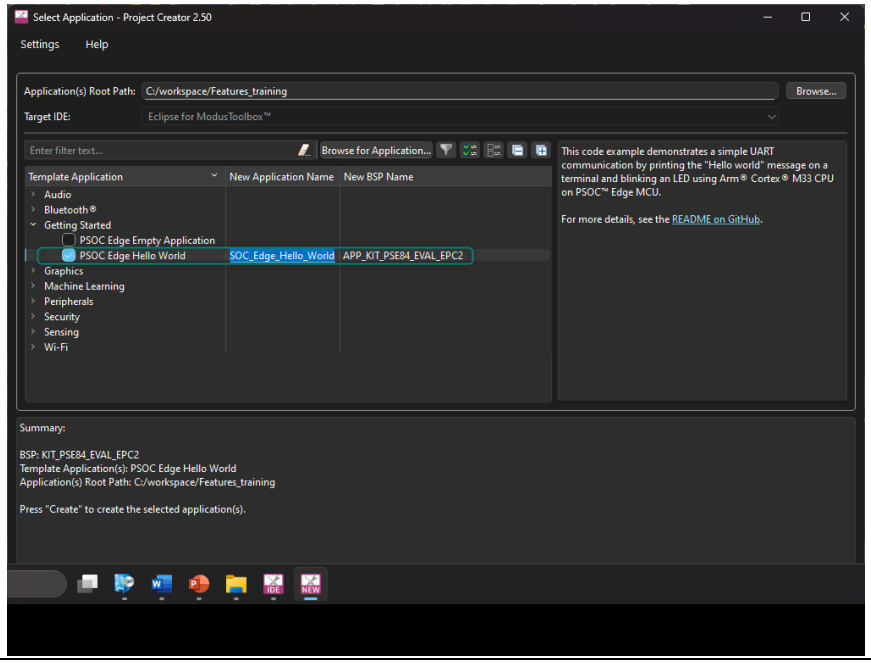


Figure 27 Selecting application

6. A new project is created, and with all the files visible in the **Project Explorer** window.

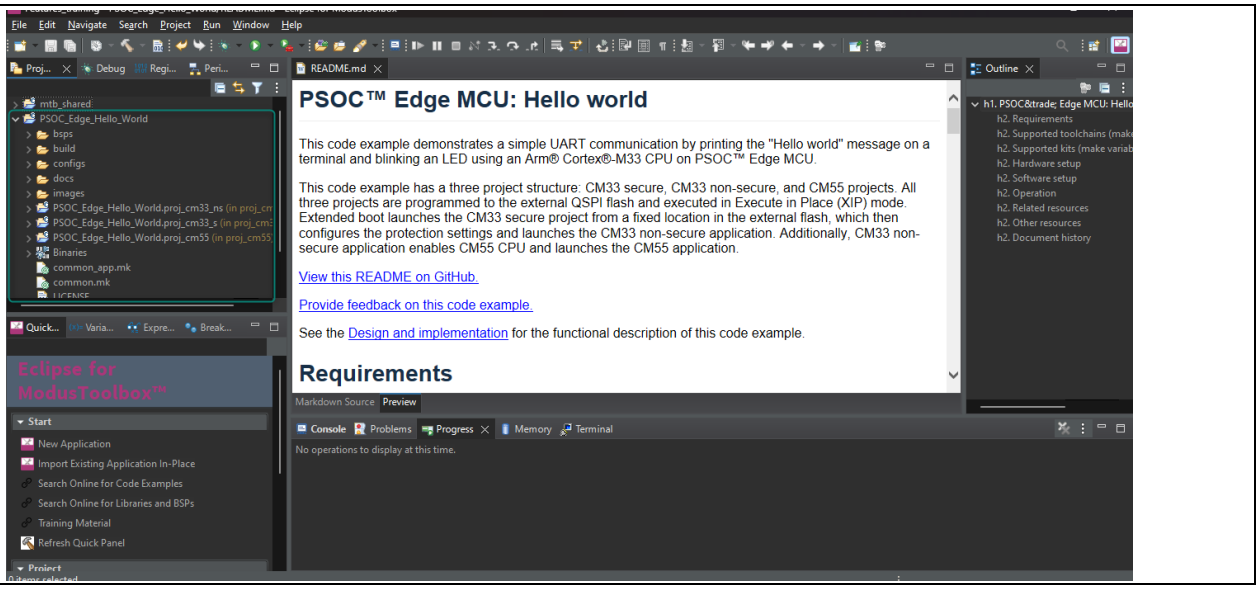


Figure 28 New application created

6 Appendix B: KIT_PSE84_EVAL details

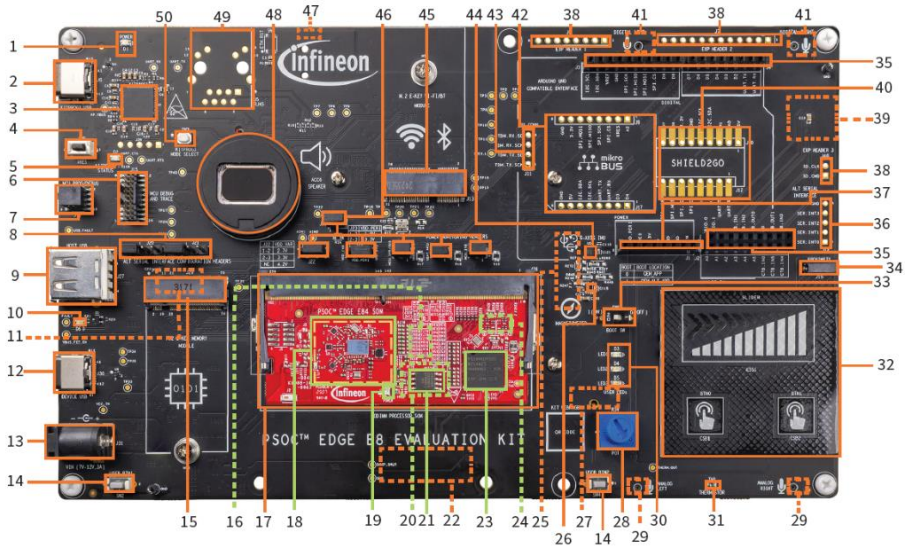


Figure 29 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-01-15	Initial release

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-26

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com